
Lab - Week 8

Classical Game Theory (Nash Equilibrium)

The purpose of this lab is to give you gain practice with determining outcomes of normal form games for pure and mixed strategies.

1. Dominated Strategies

Tasks

- 1.1 Define formally what a dominated strategy is.
- 1.2 Explain why a dominated strategy is an important concept.
- 1.3 Find out what the difference is between strictly dominated and weakly dominated. Explain!
- 1.4 We have defined the game *Morra* in the lecture. Which strategies are dominated in *Morra*? For this, recall that the payoff matrix for (our version of *Morra*) is:

$$P = \left(\begin{array}{c|c|c|c} 0 & 2 & -3 & 0 \\ \hline -2 & 0 & 0 & 3 \\ \hline 3 & 0 & 0 & -4 \\ \hline 0 & -3 & 4 & 0 \end{array} \right);$$

- 1.5 For each of the bi-matrix games below, name the dominated strategies. Which of the dominated strategies are weakly dominated, which strictly?

$$A = \left(\begin{array}{c|c} 2 & 1 \\ \hline 1 & 1 \end{array} \right); \quad B = \left(\begin{array}{c|c} 1 & 1 \\ \hline 1 & 3 \end{array} \right);$$

$$A = \left(\begin{array}{c|c|c|c} 2 & 1 & 3 & 17 \\ \hline 27 & 3 & 1 & 1 \\ \hline 4 & 6 & 7 & 18 \end{array} \right); \quad B = \left(\begin{array}{c|c|c|c} 11 & 9 & 10 & 22 \\ \hline 0 & 1 & 1 & 0 \\ \hline 2 & 10 & 12 & 0 \end{array} \right)$$

$$A = \left(\begin{array}{c|c|c} 3 & 3 & 2 \\ \hline 2 & 1 & 3 \end{array} \right); \quad B = \left(\begin{array}{c|c|c} 2 & 1 & 3 \\ \hline 2 & 3 & 2 \end{array} \right);$$

- 1.6 [optional - *at home unless time permits*] Write a (Python) program to automatically determine the pareto optimal outcomes for a bi-matrix game.
-

2. Nash Equilibrium

Nash Equilibrium in Pure Strategies

A pure Nash Equilibrium contains only one strategy for each player.

Tasks

- 2.1 Does the game in bi-matrix notation below have a pure Nash equilibrium? Give the equilibrium strategy profile or show why there is no pure Nash equilibrium.

$$A = \left(\begin{array}{c|c} 1 & 2 \\ \hline 2 & 1 \end{array} \right); \quad B = \left(\begin{array}{c|c} 4 & 2 \\ \hline 3 & 6 \end{array} \right);$$

2.2 Does the game from 4.1 have a mixed Nash equilibrium? Give the equilibrium point or give reasons why mixed Nash does not exist.

2.3 Define a two-player zero sum game that has a pure Nash equilibrium.

3. Finding Nash Equilibria by Linear Programming

Some games have an equilibrium only in mixed strategies. In general, this is computationally hard to find, but it can be easily done for 2-person zero sum games. Below you need to find such equilibria using linear programming. You can do this either in Python or in Mathematica (or, of course, on a piece of paper, but I don't recommend that!).

Finally solving *Morra*

We will now finally solve *Morra*. Recall that the payoff matrix for (our version of *Morra*) is:

$$P = \left(\begin{array}{c|c|c|c} 0 & 2 & -3 & 0 \\ \hline -2 & 0 & 0 & 3 \\ \hline 3 & 0 & 0 & -4 \\ \hline 0 & -3 & 4 & 0 \end{array} \right);$$

Tasks

3.1 Use the linear programming support of your preferred system (Python, Mathematica, ...) to determine the mixed Nash equilibrium for *Morra* exactly (or up to numerical precision in Python). As normally we will want a minimisation problem, the most direct way is to write down the system for Player B.

3.2 Write a Monte Carlo simulation for *Morra* that takes a mixed strategy as input and determines the expected utility assuming that both players play this strategy (using a fixed number of iterations given as a third parameter).

3.3 Using your solution from 1.2 verify whether *Morra* is a fair game.

Some coding tips you may need:

If you want to use *Mathematica* for this, you can basically just directly write down the problem as you normally would. You should use the built-in *LinearOptimization* function, which is fully documented here: <https://reference.wolfram.com/language/ref/LinearOptimization.html>.

Assume you want to Minimize $x+y$ subject to $x + 2y == 3; x \geq 0; y \geq 0$

```
In[ ]:= res = LinearOptimization[
  x + y,
  {x + 2 y == 3, x ≥ 0, y ≥ 0},
  {x, y}
]
```

Out[*]=

$$\left\{ x \rightarrow 0, y \rightarrow \frac{3}{2} \right\}$$

The first argument is the objective, the second one the set of constraints (mind the double equal sign!) and the third one the list of variables to optimise for.

If you want to use *Python* for this, the simplest way is to use the *linprog* linear programming routines in *scipy.optimize*. The full documentation is here: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>

You may prefer to start from a simple example. There are many places, where you can find these on the web, for example: <https://towardsdatascience.com/linear-programming-with-python-db7742b91cb>.

Of course, whichever system you end up using, you should normally really write this in matrix notation. To do so, you need to generate matrices to write an LP in standard form. In standard form we split the inequalities and the equalities into two different sets of matrices, which results in the form

minimize $F \cdot X$ subject to $A \cdot X + B \geq 0$ and $A1 \cdot X + B1 = 0$

so for the above :

In[*]:= **F = {1, 1};**

In[*]:= **A = {{1, 0}, {0, 1}};**

In[*]:= **B = {0, 0};**

In[*]:= **A1 = {{1, 2}};**

In[*]:= **B1 = {-3};**

In[*]:= **LinearOptimization[F, {A, B}, {A1, B1}]**

Out[*]=

$$\left\{ 0, \frac{3}{2} \right\}$$